Image Encoder-Decoders

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Describe a task where the output is an image the same size as the input (segmentation)
- Explain how an encoder-decoder CNN shrinks an image and then grows it back
- Compute a transposed convolution and its output size by hand
- Explain the same bottleneck problem as in sequence to sequence, and how skip connections address it
- Compare the encoder-decoder for sequences with the encoder-decoder for images

Images in, images out

One label per pixel

From one label to a label per pixel

So far a CNN has turned a whole image into **one** answer: a class label or a feature vector.

- “Is this a cat?” needs one answer per image
- “Which pixels are the cat?” needs an answer for **every pixel**. This task is called **segmentation**
- The output now has to be as big as the input. Does a CNN, which keeps shrinking its maps, have anything for that?

The question:

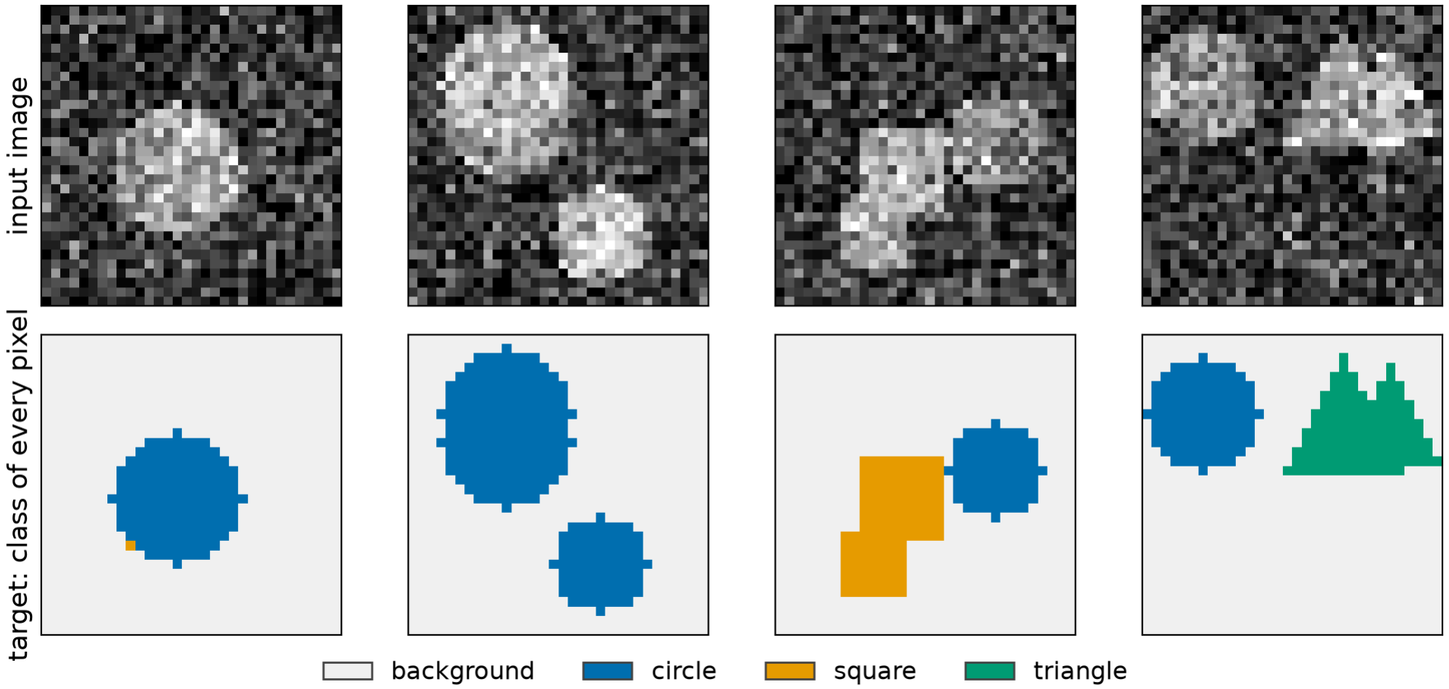
How do we get a prediction at every pixel, instead of one prediction per image?

The Big Idea

Shrink the image into a small summary with a CNN, then grow it back to full size, predicting a class at every pixel.

- The shrinking half is the CNN we already know
- This is the same shape as sequence to sequence: an encoder makes a summary, and a decoder writes the output from it

The task: segmentation

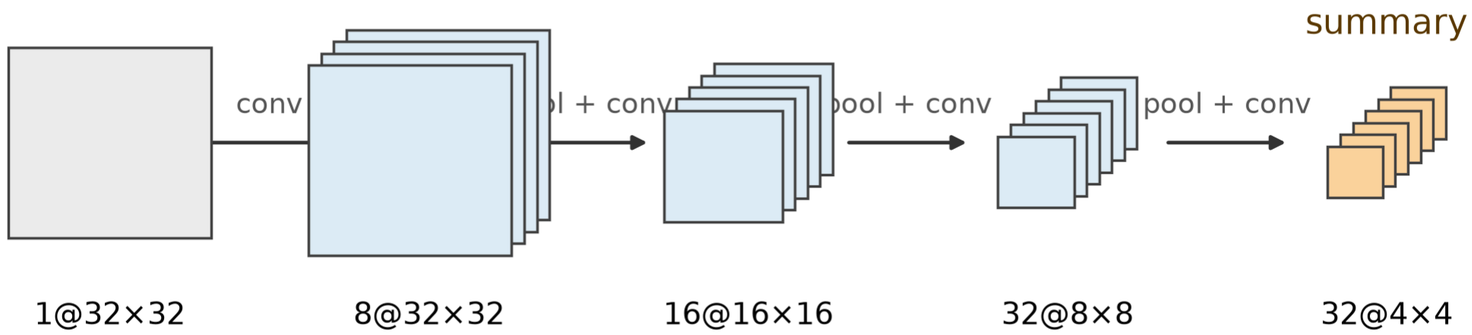


- A toy task with real training: 32×32 images, a few circles, squares and triangles at random brightness, on a noisy background
- The model sees the top row and has to produce the bottom row

Shrink, then grow

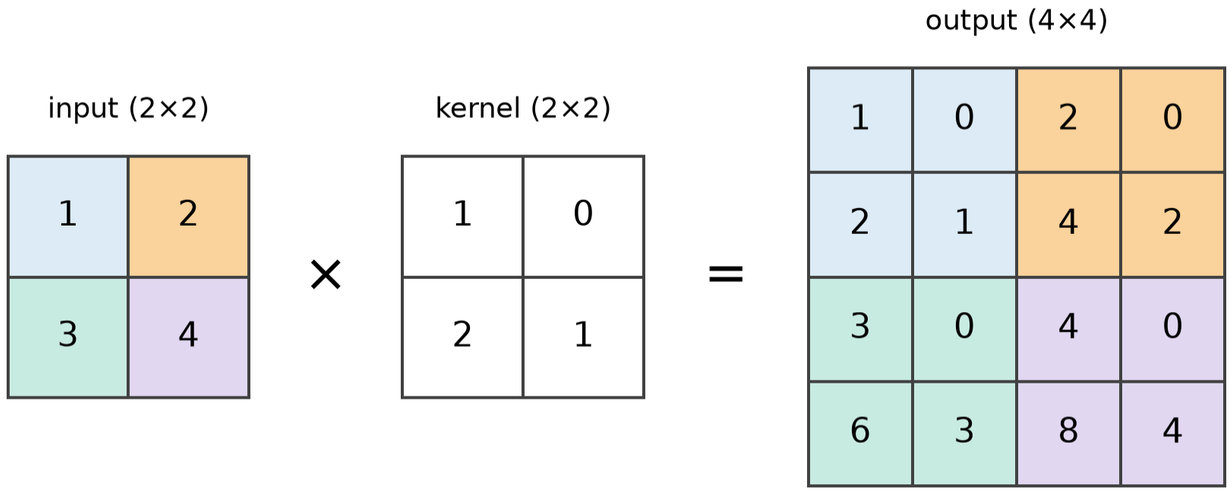
An encoder we know, and a decoder that upsamples

Encoder: a CNN we know



- Each stack is a feature map: **8@32×32** means 8 channels, each 32×32. Pooling **halves** the height and width, and the convolutions add channels
- The last map is the **summary**. Each of its 16 cells covers an 8×8 patch of the original image

Transposed convolution: growing a map



- Each input value **scales a copy of the kernel**, and the copies are stamped side by side (stride 2). For example, 3 × kernel gives the bottom-left block
- This kernel is learned, as in any convolution. This one layer takes a 2×2 map to a 4×4 map

Transposed convolution: the size

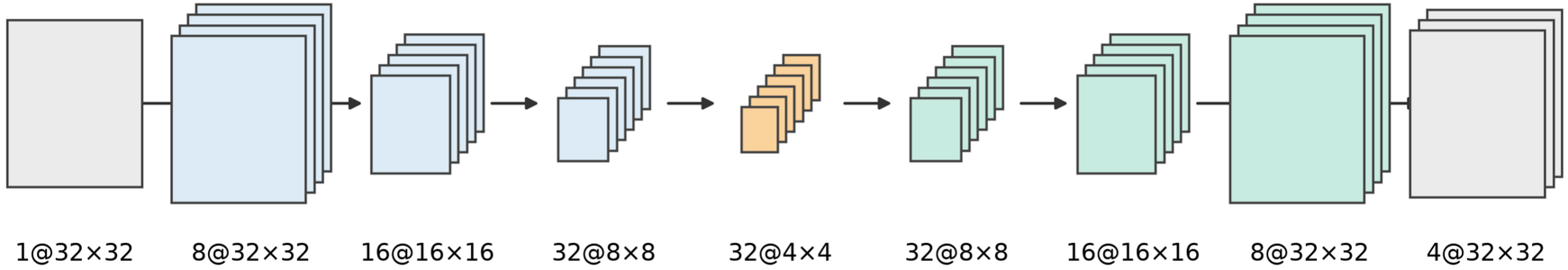
$$\text{output size} = (n-1)\cdot s + k$$

- n: size of the input map
- s: stride, the gap between stamps
- k: size of the kernel

- The example: $(2-1)\cdot 2+2=4$
- Our decoder: $(4-1)\cdot 2+2=8$, and likewise 8 to 16 and 16 to 32

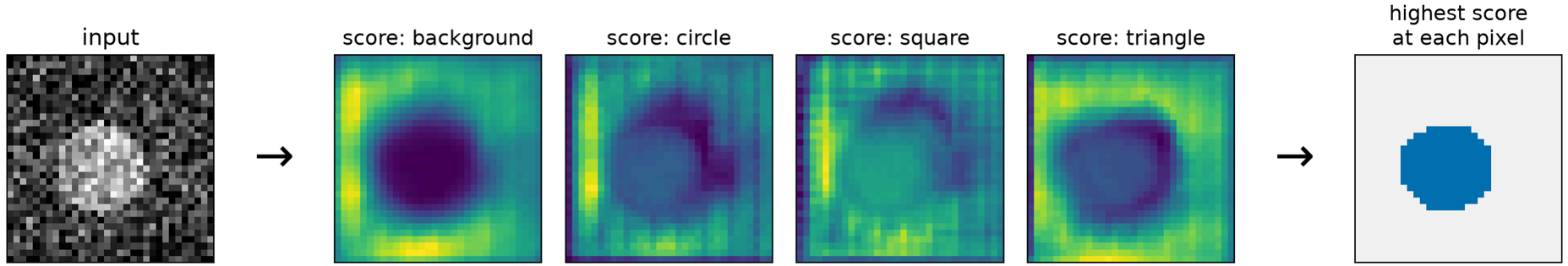
- With $k=s$ the stamps just touch. With $k>s$ they overlap and the overlapping values **add**
- It is a convolution run in reverse: a stride-2 convolution halves a map, a stride-2 transposed convolution doubles it

The whole network



- Encoder: three conv + pool steps shrink 32×32 to 4×4. Decoder: three transposed convolutions grow it back to 32×32
- The output has **4 channels**, one score for each class, at every pixel

A prediction at every pixel



- The network outputs **one score map per class**, each as big as the image: 4 × 32 × 32 numbers. Brighter means a higher score
- At every pixel we take the class with the highest score, and that gives the predicted mask on the right

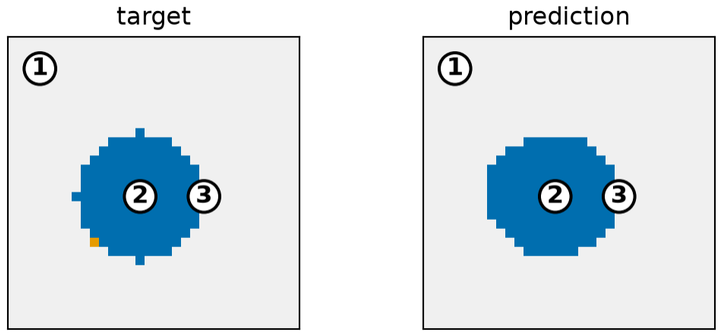
The loss at every pixel

$$L = -\frac{1}{HW} \sum_{i,j} \log p_{ij}(y_{ij})$$

- p_{ij}: softmax of the 4 scores at pixel (i, j)
- y_{ij}: the true class of that pixel
- $-\log p_{ij}(y_{ij})$: cross-entropy at one pixel
- Average over all $H \times W$ = 1,024 pixels

- It is the same cross-entropy as always, just once per pixel. The RNN had one loss per step, here there is one loss per pixel

The loss at three pixels



Pixel	True class	Scores	Softmax	Loss
1	background	27.7, 5.7, 2.0, 7.2	1.00, 0.00, 0.00, 0.00	0.00
2	circle	-3.6, 2.2, 1.3, 0.8	0.00, 0.61, 0.24, 0.14	0.49
3	circle	0.6, 0.1, -1.4, -1.5	0.53, 0.33, 0.07, 0.07	1.12

- Class order: background, circle, square, triangle. Pixel 3 is on an edge and wrong, so its loss is high. The image's loss is the **average** over its 1,024 pixels

The segmentation network in PyTorch

```
def conv_block(c_in, c_out):                    # two 3x3 convs, same size
    return nn.Sequential(nn.Conv2d(c_in, c_out, 3, padding=1), nn.ReLU(),
                         nn.Conv2d(c_out, c_out, 3, padding=1), nn.ReLU())

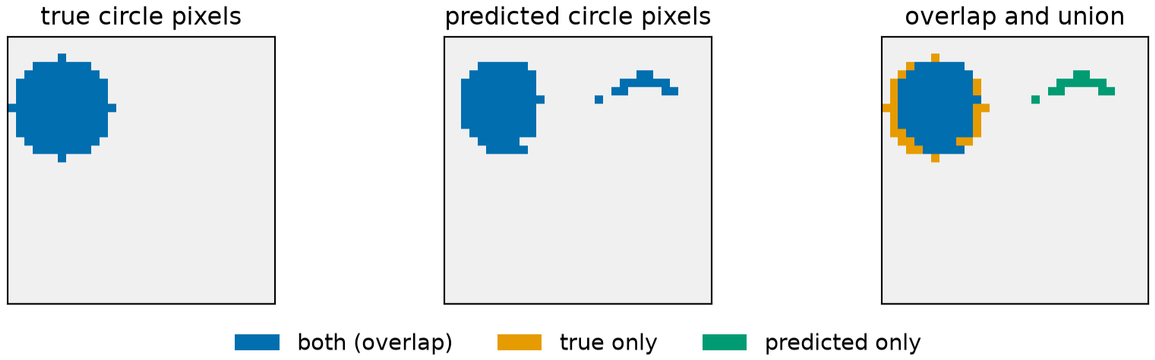
def down(c_in, c_out):                          # halve the height and width
    return nn.Sequential(nn.MaxPool2d(2), conv_block(c_in, c_out))

def up(c_in, c_out):                            # double the height and width
    return nn.Sequential(nn.ConvTranspose2d(c_in, c_out, 2, stride=2),
                         conv_block(c_out, c_out))

encoder = nn.Sequential(conv_block(1, 8), down(8, 16), down(16, 32), down(32, 32))  # 32x32 -> 4x4
decoder = nn.Sequential(up(32, 32), up(32, 16), up(16, 8))  # 4x4 -> 32x32
model = nn.Sequential(encoder, decoder, nn.Conv2d(8, 4, 1))  # 4 scores per pixel
```

- The loss is `F.cross_entropy(model(images), masks)`: scores of shape (N, 4, 32, 32) against integer classes of shape (N, 32, 32). It averages over every pixel

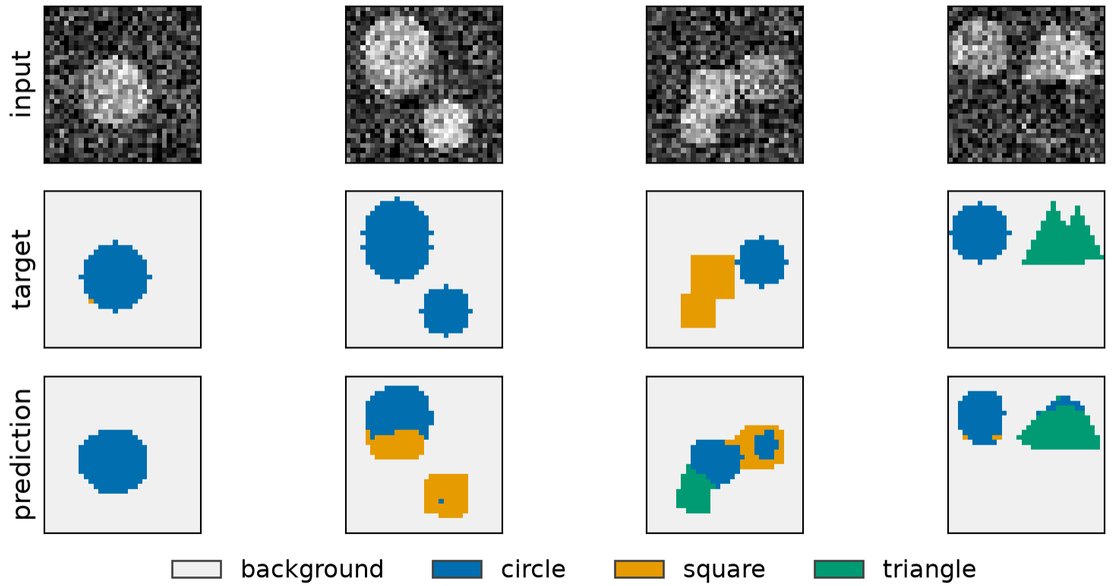
Scoring a segmentation



$$\text{IoU} = \frac{\text{pixels in both}}{\text{pixels in either}} = \frac{87}{126} = 0.69$$

- IoU (intersection over union) scores **one class**: 1.0 is a perfect match and 0 is no overlap. **Mean IoU** averages it over the classes
- Plain pixel accuracy is dominated by the large background, so IoU is the stricter score

Predictions on images it never saw

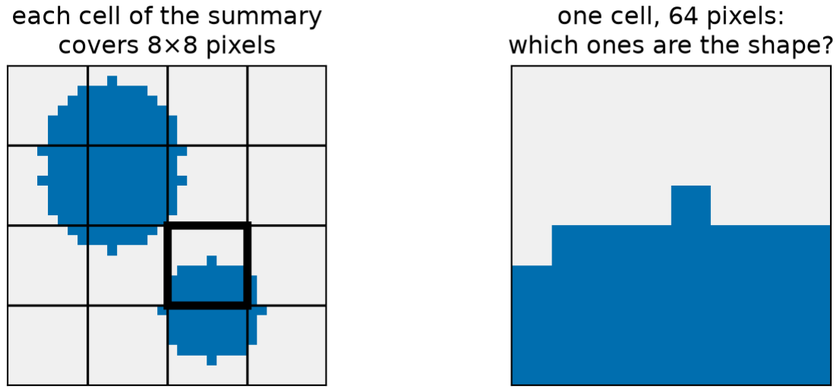


- Trained on 1,500 images, then tested on 300 new ones: **91%** of pixels are right (90% to 92% over three training runs)
- It finds the shapes, but the edges are ragged and corners are lost. Mean IoU is only **0.63**

Recovering lost detail

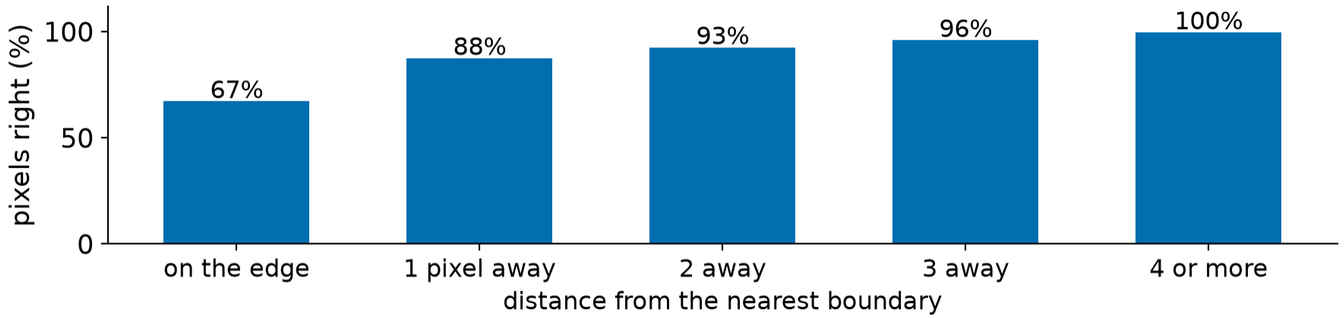
The summary is too coarse for exact edges

The problem: a coarse summary



- The summary has 16 cells, each describing a whole 8×8 patch. A cell can say “there is part of a circle here”, but not exactly which of its 64 pixels
- Earlier, one state could not hold a long list. Here one small map cannot hold exact edge positions

Where does it go wrong?



- Far from a boundary the model is nearly always right. **On** a boundary it is wrong a third of the time

The question:

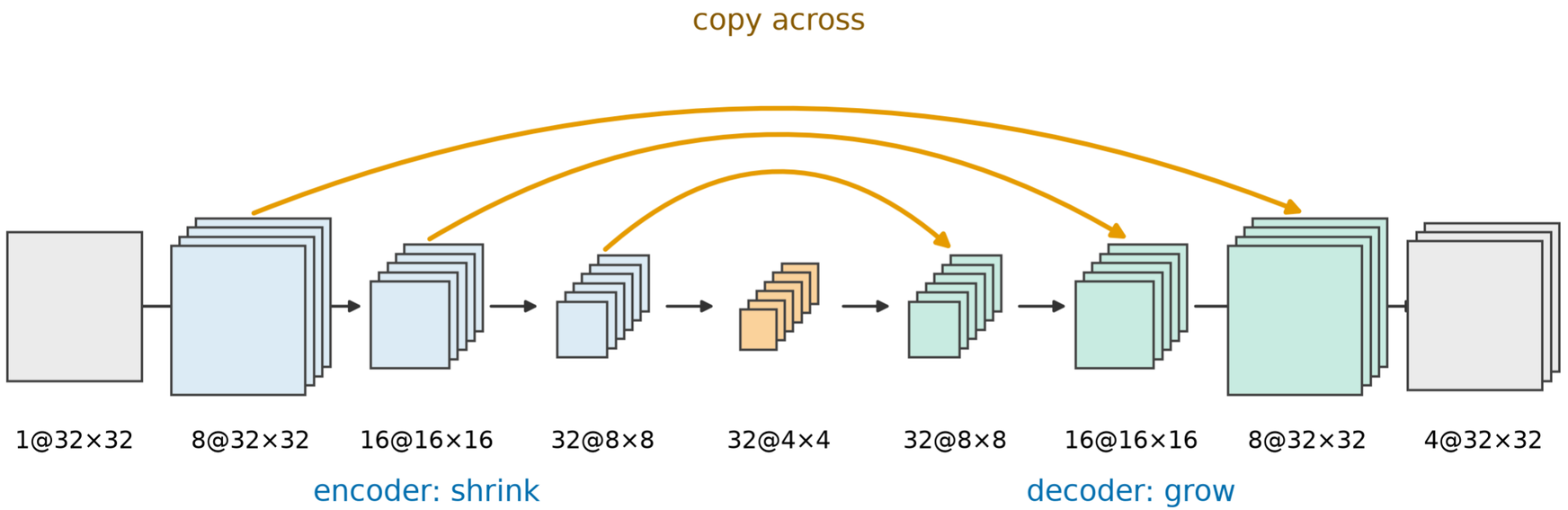
How can the decoder recover detail the summary lost?

The Big Idea

Let the decoder look back at the encoder's feature maps, not only at the final summary.

- The encoder already computed detailed 32×32, 16×16 and 8×8 maps on the way down. Hand them across
- The worry from sequence to sequence again: one summary cannot carry everything

Skip connections



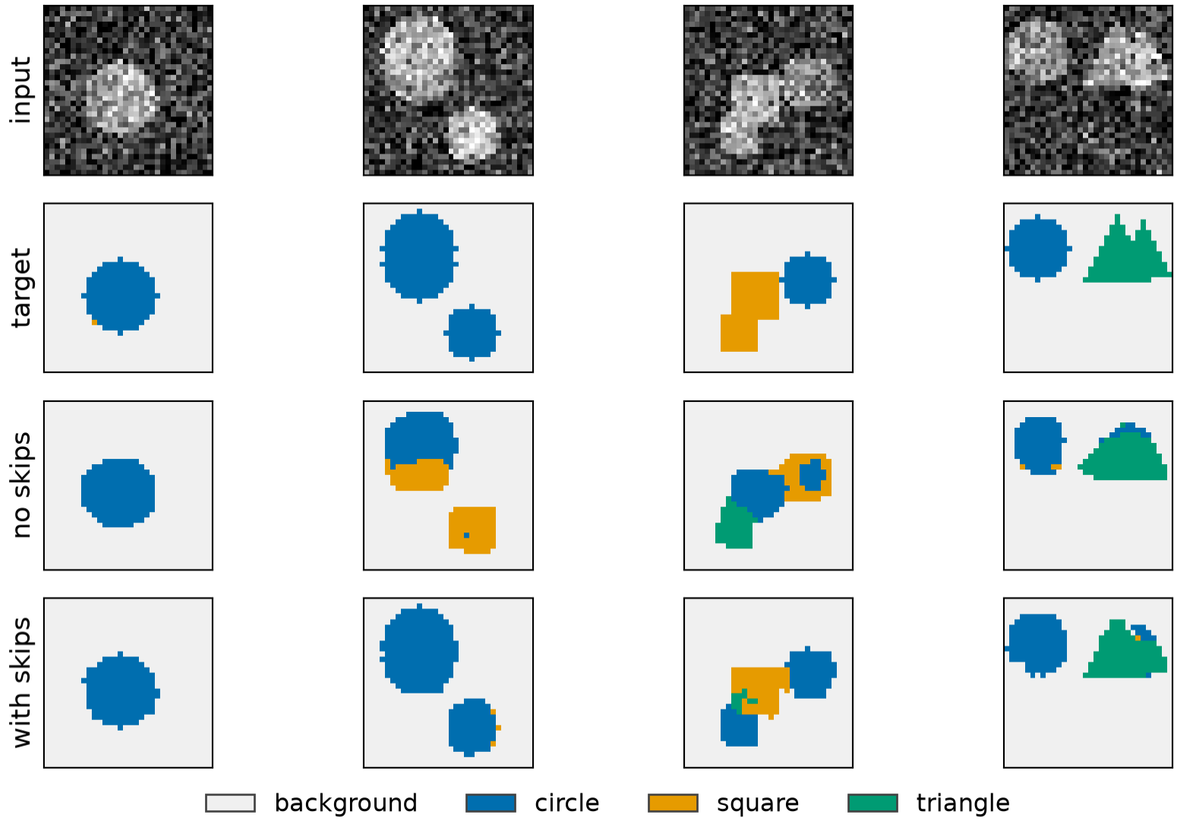
- Each decoder level also receives the **encoder map of the same size** (orange). The two are stacked along the channel dimension (**concatenated**)
- The drawing is U-shaped, so this network is called a **U-Net**

Tracing the shapes

Decoder level	Upsampled map	Encoder map copied across	Stacked	After the conv block
8×8	8×8×32	8×8×32	8×8×**64**	8×8×32
16×16	16×16×16	16×16×16	16×16×**32**	16×16×16
32×32	32×32×8	32×32×8	32×32×**16**	32×32×8

- Concatenating **doubles the channels** at each level, so each decoder conv block takes twice as many input channels as before
- The map sizes match, so the maps can be stacked. The encoder's 4×4 summary still starts the decoder
- In PyTorch, the stacking is `torch.cat([self.up(x), skip], dim=1)`, and `forward` keeps each encoder map to hand across. Same loss, same optimizer

With and without skips



- The same two models trained the same way, one with skip connections. With skips the boundaries are sharper and the triangle corners come back

The numbers

Model	Pixels right	Mean IoU	Pixels on an edge right
No skips	91.2% (90.4% to 92.2%)	0.63 (0.60 to 0.66)	67% (66% to 69%)
With skips	95.8% (95.5% to 96.1%)	0.79 (0.77 to 0.80)	86% (85% to 87%)

- Mean over three training runs, with the range in brackets, on 300 unseen test images. Same data and training steps. The skip model has 79,684 parameters, the other 67,588
- Overall accuracy moves a little, since most pixels are easy. The **edge pixels and IoU** show the real gain, exactly where the 4×4 summary was missing detail

Skip connections, and ResNet shortcuts

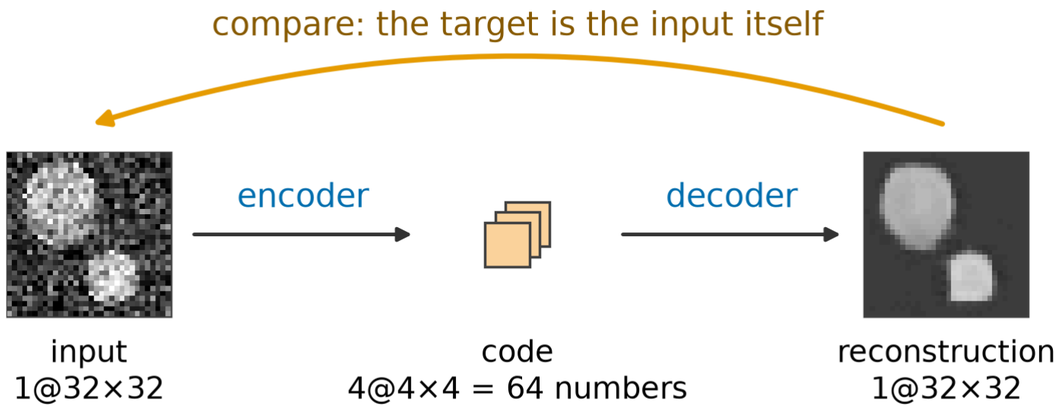
	ResNet shortcut	U-Net skip
Copies	The input of a block	An encoder map
Delivers it to	That block’s output	The decoder (same size)
Combined by	Adding	Stacking (concatenating)
Main purpose	Easier training	Recover lost detail

- Both are a fixed copy of an earlier map: nothing is chosen, and the same map always arrives at the same place
- Sequence to sequence has a similar challenge and a similar fix: let the decoder look back at the encoder. There it **chooses** what to look at for each output. That idea is attention, which comes later

An Aside: Autoencoders

When the small summary is the point

Autoencoder: rebuild the input



- Same encoder and decoder as before, but now the output is the **input image again**. The loss is squared error between the two, and no labels are needed
- The code is only 64 numbers, for an image of 1,024 pixels. To rebuild the image from it, the code has to keep what matters. It is a learned summary of the image, like the embeddings we have seen

The autoencoder in PyTorch

```
# conv_block, down and up: the helpers from the segmentation network
encoder = nn.Sequential(conv_block(1, 8), down(8, 16), down(16, 32), down(32, 32))  # 32x32 -> 4x4
decoder = nn.Sequential(                  up(32, 32),  up(32, 16),   up(16, 8))     # 4x4 -> 32x32

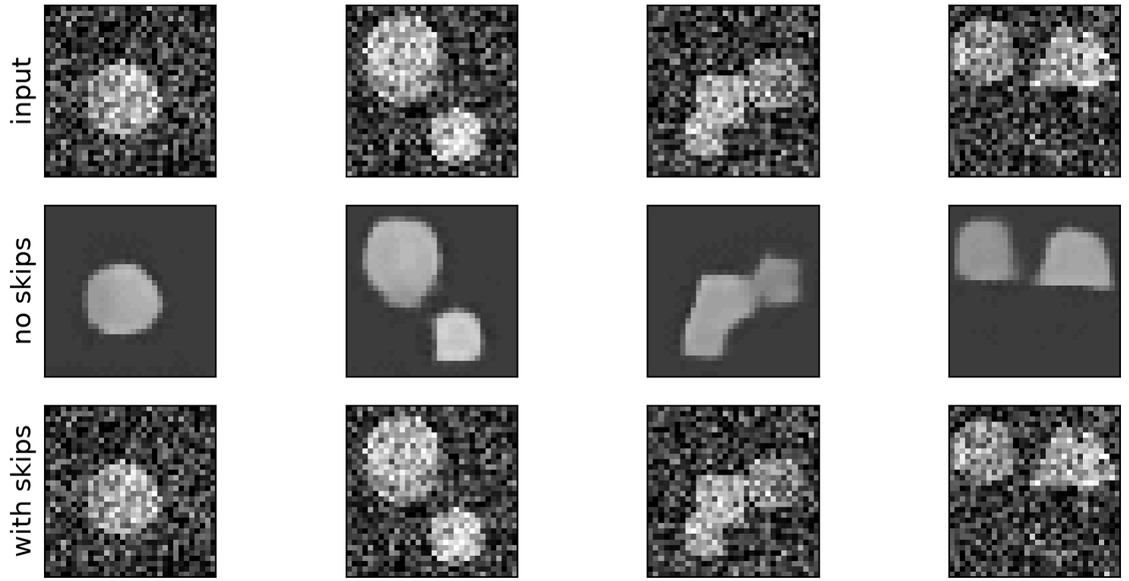
code = nn.Sequential(nn.Conv2d(32, 4, 1), nn.Conv2d(4, 32, 1)) # squeeze to 4 channels

model = nn.Sequential(
    encoder,
    code,
    decoder,
    nn.Conv2d(8, 1, 1) # 1 value per pixel
)

loss = F.mse_loss(model(x), x)  # the target is x itself
```

- Three changes from the segmentation network:
  - 1×1 convolutions squeeze 32 channels down to 4: 4 × 4 × 4 = **64 numbers**, the code
  - The output has 1 channel instead of 4
  - The loss compares the output with the input

What the code can hold



- **No skips:** the shapes come back, and the noise does not. Random noise cannot be squeezed into 64 numbers, so the code keeps the shapes
- **With skips:** a near-perfect copy of the input, noise and all. The decoder can read the input through the skips and never needs the code

Skips defeat the bottleneck

Model	Error against its input	Error against the clean image
Copy the input	0.0000	0.090
Autoencoder, no skips	0.1003	0.013
Autoencoder, with skips	0.0001	0.090

- The autoencoder never sees the clean image, yet **without skips** it lands close to it (0.013): the small code throws the noise away
- **With skips** it copies the noise too, so its error is the noise itself (0.090). Skips help segmentation because we want the detail, and hurt here for the same reason
- Removing noise is also the core of **diffusion models**: a U-Net trained to denoise, run step by step from pure noise to make an image. Not covered here

Other image-to-image tasks

Same shape, different task

Same network, different output

Task	Input	Output at each pixel	Loss
Segmentation	A photo	A class	Cross-entropy
Colorization	A grayscale image	Two color values	Squared error
Denoising	A noisy image	The clean value	Squared error
Depth estimation	A photo	A distance	Squared error

- The encoder-decoder stays. Only the **number of output channels** and the **loss** change, as with regression versus classification earlier
- Denoising is the autoencoder from before with one change: the target is the clean image, not the input

Summary

Recap: using a CNN

Use	Output	What we use	Loss
Classify an image	One class	The last features, flattened	Cross-entropy
Pretrained features	One vector	The last features, as an embedding	Whatever the new task needs
Segment an image	A class at every pixel	Encoder, decoder, and skips	Cross-entropy at every pixel
Predict a value per pixel	A number at every pixel	Encoder, decoder, and skips	Squared error
Autoencode an image	The image again	Encoder, small code, decoder	Squared error

Recap: key ideas

- A CNN can produce an **image-sized output**: an encoder shrinks the image, and a decoder of **transposed convolutions** grows it back
- Segmentation gives one class score per pixel, and the loss is cross-entropy averaged over the pixels
- The small summary loses detail, so boundaries blur. **Skip connections** copy encoder maps to the decoder at matching sizes, and sharpened the boundaries on our toy
- Sequence to sequence has a similar bottleneck and a similar fix: let the decoder look back at the encoder
- An **autoencoder** is an encoder-decoder whose target is its own input. Its small code is the useful part, and skips would defeat it

Where this goes next

- Attention: let a decoder look back at **every** encoder step, weighted by how relevant each is. Skip connections are a fixed version of this: the same map, at the same position, with nothing chosen
- Attention without any recurrence leads to the transformer
- Not covered here: object detection, instance segmentation, generating images with diffusion models, and applying transformers to images